

PHASE 07 / 07

Production SQL

Real-time scenarios, views, stored procedures, user-defined functions and a roadmap for what to learn next as a data engineer.

Scenarios | Views | Stored Procedures | Functions | Next Steps

The last phase turns individual queries into reusable, automatable building blocks, then shows you where this knowledge plugs into a real data-engineering career.

CONTENTS

Inside this phase

Real-Time Scenarios 3
- A method for scenario questions 3
- Scenario 1 · The n-th value per group 4
- Scenario 2 · Removing duplicates 6
- Scenario 3 · Compare with the previous or next row 7
- Scenario 4 · Consecutive days (gaps and islands) 9
- Pattern catalogue 10

Views 11
- What a view really is 11
- Creating the course's dedup_view 12
- Why people use views 13
- Can I INSERT / UPDATE through a view? 14
- View vs a stored summary table 15

Stored Procedures 17
- The anatomy of a call 17
- DELIMITER: why it is needed 18
- The course's first procedure 18
- Parameters: IN, OUT, INOUT 19
- Variables and control flow 20
- Error handling 21
- Transactions inside a procedure 22
- An ETL-style procedure: idempotent daily refresh 24

User-Defined Functions 27
- Procedure or function? 27
- The course's function 28
- Function characteristics 29
- Useful everyday functions 30
- Performance: functions run once per row 31

The Next Steps 32
- Where SQL sits in a data platform 32
- A taste of indexes and EXPLAIN 33
- Capstone: a mini ELT pipeline 34
- Your roadmap 37
- Interview question bank 37
- Master cheat sheet 39
- Course topic coverage map 40

Phase 7 Recap & Practice 41
- Cheat sheet 41
- Check yourself 41
- Practice lab 42
- Answer key 42

7.1 PART 7.1 · PROBLEMS YOU WILL ACTUALLY BE ASKED

Real-Time Scenarios

This part puts Phases 3 to 6 to work on the patterns that appear again and again in interviews and in real pipelines: *the n-th highest value*, *removing duplicates*, *comparing with the previous row*, and *finding streaks*. Learn the pattern, then you can recognize it in disguise.

What you will learn

- Find the n-th value per group with `DENSE_RANK`
- Remove duplicates deterministically with `ROW_NUMBER`
- Compare consecutive rows with `LAG` and `LEAD`
- Solve 'consecutive days' problems (gaps and islands)

Why it matters for data engineering. Duplicates arrive from retries and re-sent files, 'latest record per key' is the core of change-data-capture, and period-over-period comparison is the most common business metric. These four patterns cover a large share of everyday warehouse SQL.

A method for scenario questions

1. **Name the grain**
   What does **one output row** represent? 'One per category'? 'One per customer'? Writing it down prevents wrong GROUP BYs.

2. **Pick the tool**
   Collapse rows → `GROUP BY`. Keep rows and compare/rank → **window function**. Multi-step → CTE.

3. **Think about ties and NULLs**
   What if two rows share the top value? What if the first row has no previous row? Choose `RANK` vs `DENSE_RANK` vs `ROW_NUMBER`, and a `LAG` default, on purpose.

4. **Test on a tiny table**
   Hand-build 5 to 8 rows including the awkward cases (ties, gaps, duplicates), run, and compare with the answer you worked out on paper.

Scenario 1 · The n-th value per group

Question: find the 5th most expensive product in every category. The course answer ranks with `DENSE_RANK` inside each category (partition), then filters in an outer query.

PARTITION: Books

unit_price	ranking
978.88	1
975.35	2
962.74	3
941.43	4
940.38	5
922.31	6

PARTITION: Toys

unit_price	ranking
999.04	1
967.14	2
952.58	3
932.85	4
931.1	5
905.0	6

Outer query

WHERE ranking = 5 keeps one row per partition: the 5th most expensive.

(Illustrative values: the real run is below.)

Figure 1. Rank inside each category with `DENSE_RANK() … ORDER BY unit_price DESC`, then keep ranking = 5.

SQL — The course's Scenario 1, run for real: the 5th highest price in each of the 6 categories

```sql
SELECT subquery.*
FROM (
    SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY unit_price DESC) AS ranking
    FROM   dim_product
) subquery
WHERE subquery.ranking = 5
ORDER BY subquery.category;
```

RESULT — 6 ROWS

product_key	product_id	product_name	category	brand	unit_price	launch_date	ranking
251	PROD0251	Less Other	Books	BrandD	940.38	2024-05-08	5
102	PROD0102	Address Until	Clothing	BrandA	954.82	2023-12-21	5
484	PROD0484	Available Take	Electronics	BrandB	931.46	2021-11-08	5
434	PROD0434	Fact Nor	Home & Kitchen	BrandA	946.42	2023-08-17	5
432	PROD0432	Art Left	Sports	BrandC	892.75	2023-03-25	5
457	PROD0457	Step Situation	Toys	BrandC	931.90	2025-09-24	5

Why DENSE_RANK? The tie test

Imagine salaries where two people tie for second place. Which function gives the *second-highest salary*?

SQL — Bina and Chirag both earn 90, the 2nd-highest salary in Eng

```sql
SELECT name, dept, salary,
       ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn,
       RANK()       OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS dense
FROM   emp
ORDER  BY dept, salary DESC, name;
```

RESULT — 6 ROWS

name	dept	salary	rn	rnk	dense
Aarav	Eng	100	1	1	1
Bina	Eng	90	2	2	2
Chirag	Eng	90	3	2	2
Dev	Eng	70	4	4	3
Esha	Ops	80	1	1	1
Farid	Ops	60	2	2	2

ROW_NUMBER = 2
returns: Bina only
misses Chirag

RANK = 2
returns: Bina, Chirag
fine here, but RANK = 3 would return nobody

DENSE_RANK = 2
returns: Bina, Chirag
correct, and stable for every n

Figure 2. What each ranking function returns for 'the second-highest salary' in Eng.

INTERVIEW ANGLE

"Second highest salary" has at least three answers: `DENSE_RANK = 2`; `ORDER BY salary DESC LIMIT 1 OFFSET 1` over `SELECT DISTINCT`; or a scalar subquery `MAX(salary) WHERE salary < (SELECT MAX(salary) …)`. Know the window one best: it generalizes to **n-th per group**.

Scenario 2 · Removing duplicates

Duplicates are the most common data-quality problem: a file loaded twice, an API retry, a missing unique key. The course's raw customers table contains repeated rows.

SQL — 6 rows, but only 3 distinct customers

```sql
SELECT * FROM customers_raw ORDER BY id;
```

RESULT — 6 ROWS

id	name	email
1	asha	asha@x.com
2	bela	bela@x.com
2	bela	bela@x.com
3	chen	chen@x.com
3	chen	chen@x.com
3	chen	chen@x.com

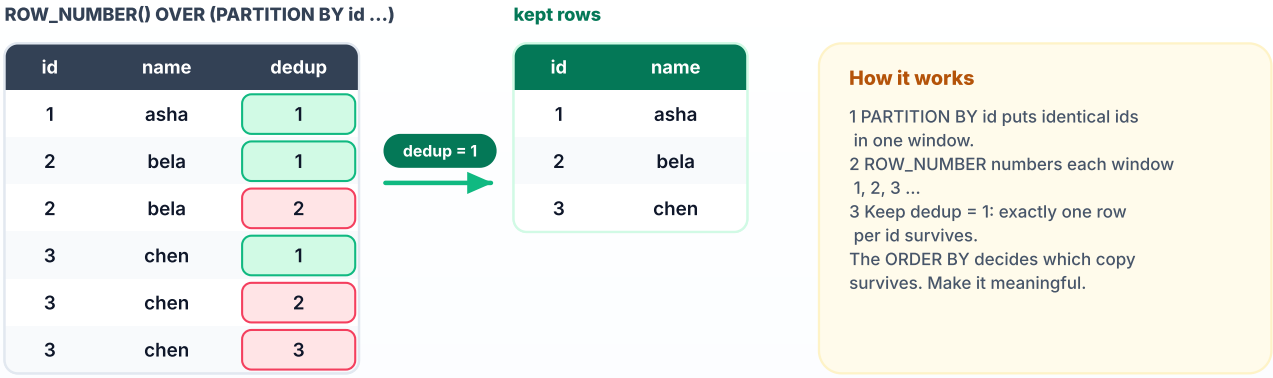


Figure 3. Numbering copies inside each id and keeping number 1.

SQL — The course's Scenario 2: exactly one row per id

```sql
SELECT subquery.id, subquery.name, subquery.email
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY id) AS dedup
    FROM   customers_raw
) subquery
WHERE dedup = 1;
```

RESULT — 3 ROWS

id	name	email
1	asha	asha@x.com
2	bela	bela@x.com
3	chen	chen@x.com

Keep the newest, not an arbitrary copy

When copies *differ* (a customer changed email), the `ORDER BY` inside the window picks the winner. Sort by the timestamp, newest first, and keep 1.

SQL — Latest record per key: the heart of change-data-capture (CDC) loads

```sql
SELECT customer_id, email, updated_at
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY updated_at DESC) AS rn
    FROM   cust_changes
) t
WHERE rn = 1
ORDER BY customer_id;
```

RESULT — 2 ROWS

customer_id	email	updated_at
1	asha@new.com	2025-03-10 14:30:00
2	bela@x.com	2025-01-15 10:00:00

DATA-ENGINEERING LENS

Idempotent loads. In a pipeline you rarely `DELETE` duplicates in place. You build a clean table (or view) *from* the raw one with this query, so the raw data stays untouched and the job can be re-run safely. The pattern is called **dedup-on-read**, and a `dedup_view` (Part 7.2) is exactly that.

Swapping in the clean table

SQL — Materialize the clean rows, then add the key that prevents the problem

```sql
CREATE TABLE customers_clean AS
SELECT id, name, email
FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY id) AS dedup FROM customers_raw) t
WHERE dedup = 1;

ALTER TABLE customers_clean ADD PRIMARY KEY (id);   -- now duplicates can never come back
```

SQL

```sql
SELECT COUNT(*) AS rows_now FROM customers_clean;
```

RESULT — 1 ROW

rows_now
3

Scenario 3 · Compare with the previous or next row

Question: for each day, show the temperature, yesterday's, the day before's, and tomorrow's. This is `LAG` and `LEAD` from Phase 6, using the course's weather table. The third argument `0` is the default for rows that have no

neighbor.

SQL — The course's Scenario 3. The 0s at the edges come from the default argument.

```sql
SELECT *,
       LAG(temp, 1, 0)  OVER (ORDER BY id ASC) AS prev_day_temp,
       LAG(temp, 2, 0)  OVER (ORDER BY id ASC) AS prev_2days_temp,
       LEAD(temp, 1, 0) OVER (ORDER BY id ASC) AS next_day_temp
FROM   weather;
```

RESULT — 5 ROWS

id	record_date	temp	prev_day_temp	prev_2days_temp	next_day_temp
1	2025-01-01	10	0	0	25
2	2025-01-02	25	10	0	20
3	2025-01-03	20	25	10	30
4	2025-01-04	30	20	25	28
5	2025-01-05	28	30	20	0

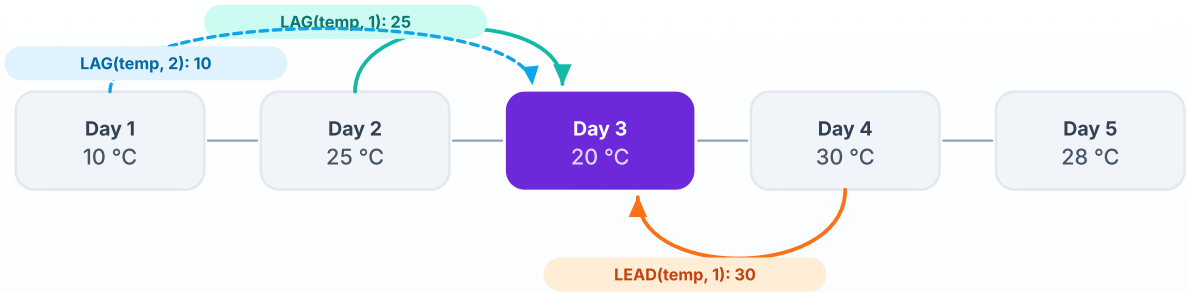


Figure 4. From day 3, `LAG(..,1)` reads day 2, `LAG(..,2)` reads day 1, and `LEAD(..,1)` reads day 4.

SQL — A typical follow-up: label each day as warmer or colder than the day before

```sql
SELECT record_date, temp,
       temp - LAG(temp) OVER (ORDER BY record_date) AS change_vs_yesterday,
       CASE WHEN temp > LAG(temp) OVER (ORDER BY record_date) THEN 'warmer'
            WHEN temp < LAG(temp) OVER (ORDER BY record_date) THEN 'colder'
            ELSE 'first day / same' END                          AS trend
FROM   weather;
```

RESULT — 5 ROWS

record_date	temp	change_vs_yesterday	trend
2025-01-01	10	*NULL*	first day / same
2025-01-02	25	15	warmer
2025-01-03	20	-5	colder
2025-01-04	30	10	warmer
2025-01-05	28	-2	colder

> ⚠ **WATCH OUT**
> `LAG` looks at the previous **row**, not the previous **date**. If a day is missing, it silently compares with whatever row comes before. For gap-free calendars, join to a date series first (the recursive calendar CTE of Phase 6).

Scenario 4 · Consecutive days (gaps and islands)

Question: for each user, find the streaks of consecutive login days. Trick: subtract a running row number from the date. Within a streak, the date and the row number both rise by one each step, so **their difference stays constant**; a gap makes it jump.

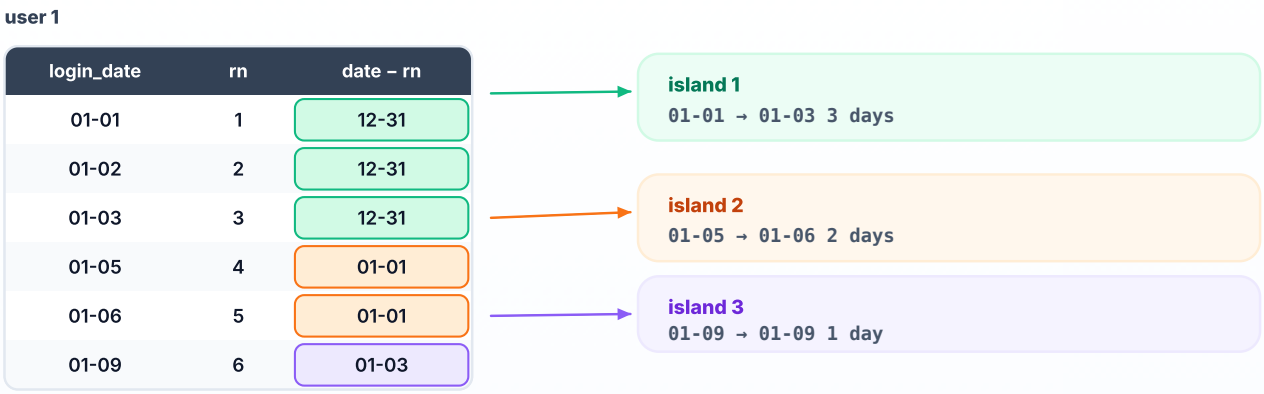


Figure 5. Same `date – rn` value = same streak. Group by that value and take MIN/MAX of the dates.

SQL — Streaks per user (user 2 has one two-day streak)

```sql
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end, COUNT(*) AS days
FROM (
    SELECT user_id, login_date,
           DATE_SUB(login_date, INTERVAL ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
login_date) DAY) AS grp
    FROM   logins
) t
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

RESULT — 4 ROWS

user_id	streak_start	streak_end	days
1	2025-01-01	2025-01-03	3
1	2025-01-05	2025-01-06	2
1	2025-01-09	2025-01-09	1
2	2025-01-01	2025-01-02	2

Pattern catalogue

When you read...	Think...	Tool
“n-th highest / top N per group”	rank in each partition, filter outside	`DENSE_RANK` / `ROW_NUMBER` + subquery
“remove duplicates”, “latest per key”	number copies, keep 1	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY ts DESC)`
“compared to previous / next / last month”	look at a neighboring row	`LAG` / `LEAD`
“running total / moving average”	cumulative frame	`SUM() OVER (ORDER BY …)`
“consecutive days / streaks”	date minus row number	`ROW_NUMBER` + `GROUP BY`
“customers with no orders”	anti-join	`NOT EXISTS` / `LEFT JOIN … IS NULL`
“percent of total”	total beside each row	`x / SUM(x) OVER ()`
“rows to columns”	conditional aggregation	`SUM(CASE WHEN …)`

7.2 PART 7.2 · SAVED QUERIES YOU CAN TREAT LIKE TABLES

Views

A **view** is a `SELECT` statement stored under a name. It holds **no data of its own**: each time you query the view, MySQL runs the stored `SELECT` against the underlying tables. You get a clean, reusable 'virtual table' without copying anything.

What you will learn

- Create, replace and drop views
- Explain what a view stores (a query) and what it does not (data)
- Use views for simplification, consistency and column-level security
- Know when a view is updatable and how `WITH CHECK OPTION` protects it
- Compare a view with a physical 'materialized' summary table

Why it matters for data engineering. Warehouses are built in layers (raw → staging → marts), and the staging layer is very often just views that rename, clean and de-duplicate raw data. Analysts query the clean interface and never touch the messy source.

What a view really is

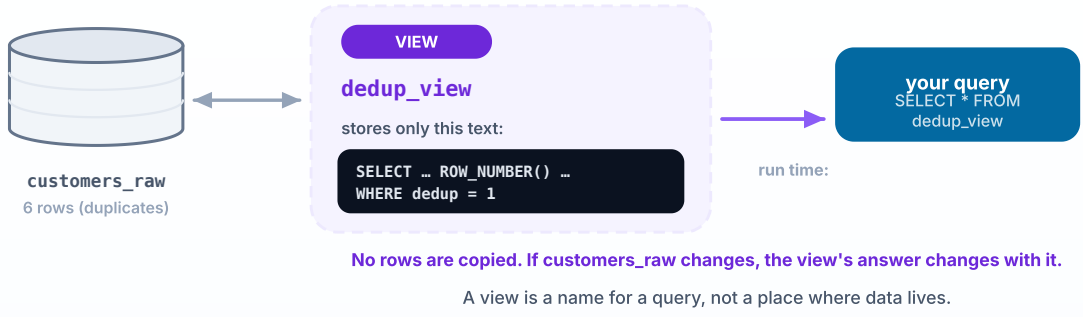


Figure 6. Querying a view expands to its stored SELECT, which runs against the base table at that moment.

Creating the course's dedup_view

Scenario 2's de-duplication query becomes a reusable object. Anyone who selects from `dedup_view` automatically gets one row per customer, without knowing about window functions.

SQL — The course's view, with explicit columns (so the helper column `dedup` is not exposed)

```sql
CREATE VIEW dedup_view AS
SELECT subquery.id, subquery.name, subquery.email
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY id) AS dedup
    FROM   customers_raw
) subquery
WHERE dedup = 1;
```

SQL — Query the view exactly like a table

```sql
SELECT * FROM dedup_view;
```

RESULT — 3 ROWS

id	name	email
1	asha	asha@x.com
2	bela	bela@x.com
3	chen	chen@x.com

SQL — Load two more raw rows (a duplicated new customer): the view reflects it instantly, still with a single row for id 4

```sql
INSERT INTO customers_raw VALUES (4, 'dev', 'dev@x.com'), (4, 'dev', 'dev@x.com');
SELECT * FROM dedup_view ORDER BY id;
```

RESULT — 4 ROWS

id	name	email
1	asha	asha@x.com
2	bela	bela@x.com
3	chen	chen@x.com
4	dev	dev@x.com

REMEMBER

A view is **live**: no refresh needed, no stale data. The price is that the full query runs every time. For cheap logic that is perfect; for heavy aggregation, consider a stored summary table (below).

Managing views

Task	Statement
Create	`CREATE VIEW v AS SELECT …;`
Change the definition	`CREATE OR REPLACE VIEW v AS SELECT …;` or `ALTER VIEW v AS …;`
See the definition	`SHOW CREATE VIEW v;`
List views	`SHOW FULL TABLES WHERE Table_type = 'VIEW';`
Remove	`DROP VIEW IF EXISTS v;`

SQL — Views live in the same namespace as tables, so names must not clash

```sql
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

RESULT — 1 ROW

Tables_in_lab	Table_type
dedup_view	VIEW

Why people use views

Simplify
Hide joins and window logic behind one name. `SELECT * FROM v_sales_clean` beats pasting a 40-line query everywhere.

Standardize
One definition of 'active customer' or 'net revenue' means every dashboard agrees. Change it once, everyone updates.

Secure
Expose only some columns or rows. Grant access to the view, not the table, so private columns stay hidden.

Stabilize
Rename or restructure the underlying tables later, and keep the view's shape the same so downstream code does not break.

Security example: hide personal columns

SQL

```sql
CREATE VIEW v_customer_public AS
SELECT customer_key, first_name, country, join_date      -- no email, no phone
FROM   sales.dim_customer;
```

SQL — Analysts see enough to work, not enough to contact people

```sql
SELECT * FROM v_customer_public ORDER BY customer_key LIMIT 3;
```

RESULT — 3 ROWS

customer_key	first_name	country	join_date
1	Antonio	Tunisia	2021-09-08
2	Todd	Djibouti	2023-08-05
3	Matthew	Guinea-Bissau	2022-03-27

SQL — Pattern only (account management is an admin task)

```sql
-- an analyst account gets the view, not the table
CREATE USER 'analyst'@'%' IDENTIFIED BY '…';
GRANT SELECT ON lab.v_customer_public TO 'analyst'@'%';
```

Can I INSERT / UPDATE through a view?

Only if MySQL can map each view row back to exactly one base-table row. A **simple** view over one table is *updatable*. A view that uses `DISTINCT`, `GROUP BY`, aggregates, window functions or `UNION` is not.

SQL — dedup_view uses a window function, so it is read-only

```sql
UPDATE dedup_view SET name = 'x' WHERE id = 1;
```

⚠ ERROR

```
ERROR 1288 (HY000): The target table dedup_view of the UPDATE is not updatable
```

WITH CHECK OPTION: keep changed rows inside the view

SQL — A simple, updatable view over `emp`

```sql
CREATE VIEW v_eng AS
SELECT id, name, dept, salary FROM emp WHERE dept = 'Eng'
WITH CHECK OPTION;
```

SQL — Allowed: the row still satisfies dept = 'Eng'

```sql
UPDATE v_eng SET salary = salary + 5 WHERE id = 4; SELECT id, name, dept, salary FROM v_eng ORDER BY id;
```

RESULT — 4 ROWS

id	name	dept	salary
1	Aarav	Eng	100
2	Bina	Eng	90
3	Chirag	Eng	90
4	Dev	Eng	75

SQL — Blocked: the new row would disappear from the view

```sql
UPDATE v_eng SET dept = 'Ops' WHERE id = 4;
```

ERROR

```
ERROR 1369 (HY000): CHECK OPTION failed 'lab.v_eng'
```

NOTE

Without `WITH CHECK OPTION` that last update would succeed, and the row would silently vanish from the view (it now belongs to `Ops`). The option makes the view enforce its own `WHERE`.

View vs a stored summary table

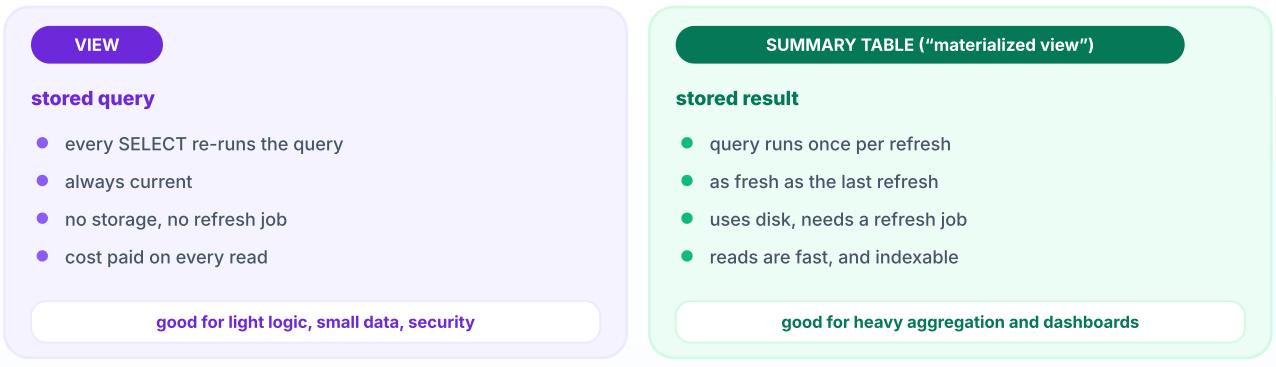


Figure 7. MySQL has no built-in materialized views (unlike PostgreSQL or Snowflake), so we emulate one with a table plus a refresh step.

SQL — A physical snapshot. Refresh it by `TRUNCATE` + `INSERT ... SELECT` (see the procedure in 7.3).

```sql
CREATE TABLE mv_category_sales AS
SELECT p.category, COUNT(*) AS orders, ROUND(SUM(f.total_amount)) AS revenue
FROM   sales.fact_sales f JOIN sales.dim_product p ON p.product_key = f.product_key
GROUP  BY p.category;
SELECT * FROM mv_category_sales ORDER BY revenue DESC;
```

RESULT — 6 ROWS

category	orders	revenue
Sports	873	2510907
Home & Kitchen	908	2474614
Clothing	884	2444861
Books	847	2350768
Toys	764	2144384
Electronics	724	2031920

DATA-ENGINEERING LENS

Layers. A typical project: *raw* tables loaded as-is → *staging* views that rename, cast and de-duplicate → *marts* (tables or incremental models) that business users query. dbt generates exactly these `CREATE VIEW` / `CREATE TABLE AS` statements from your SELECT files.

WATCH OUT

Stacked views (a view over a view over a view) become hard to debug and can be slow, because the optimizer must expand every level. Keep chains short, and remember `ORDER BY` inside a view is not guaranteed to be honored by the outer query.

7.3 PART 7.3 · PROGRAMS THAT LIVE IN THE DATABASE

Stored Procedures

A **stored procedure** is a named block of SQL statements saved on the server and run with `CALL`. Think of it as a function in a programming language: it takes parameters, can use variables, `IF` and loops, and can change data. Write the logic once, run it from anywhere.

What you will learn

- Create a procedure with `DELIMITER`, `CREATE PROCEDURE … BEGIN … END`
- Use `IN`, `OUT` and `INOUT` parameters
- Add variables, `IF` / `CASE` and loops
- Handle errors with handlers and `SIGNAL`, and wrap changes in a transaction
- Write an idempotent, ETL-style refresh procedure

Why it matters for data engineering. Procedures encapsulate repeatable, multi-step data jobs (load a day, refresh a summary, archive old rows) so applications and schedulers need only one `CALL`. Plenty of legacy and MySQL-based warehouses still run their nightly ETL this way.

The anatomy of a call

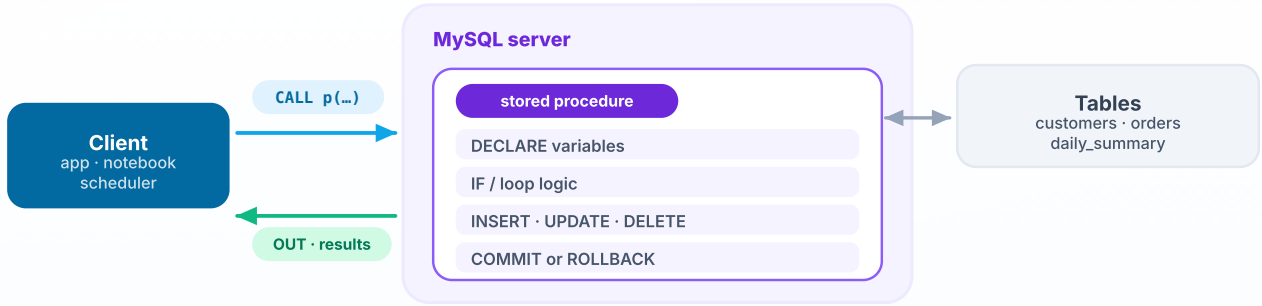


Figure 8. One round trip: the client sends a single CALL; every statement inside then runs next to the data.

DELIMITER: why it is needed

The client treats `;` as 'send this statement now'. A procedure body is *full* of semicolons, so you temporarily change the client's delimiter, define the whole procedure, then change it back.

× Default delimiter ;

```
CREATE PROCEDURE p()
BEGIN
INSERT INTO t VALUES (1);
INSERT INTO t VALUES (2);
END;
```

← cut here!

✓ DELIMITER //

```
DELIMITER //
CREATE PROCEDURE p()
BEGIN
INSERT INTO t VALUES (1);
INSERT INTO t VALUES (2);
END //
DELIMITER ;
```

Figure 9. `DELIMITER //` tells the client to send everything up to `//` as one statement. (Only the client uses it: it is not SQL.)

The course's first procedure

SQL — The procedure from the course, creating it in the `lab` database

```sql
DELIMITER //
CREATE PROCEDURE first_procedure(IN p_id INT, IN p_name CHAR(100), IN p_email CHAR(100))
BEGIN
    INSERT INTO customers
    VALUES (p_id, p_name, p_email);
END //

DELIMITER ;
```

SQL — `CALL` runs it. The new row is in the table.

```sql
CALL first_procedure(10, 'blabla', 'ejccagcjg'); SELECT * FROM customers;
```

RESULT — 1 ROW

id	name	email
10	blabla	ejccagcjg

PRO TIP

Name parameters with a prefix (`p_id`, `p_name`). If a parameter has the same name as a column, MySQL uses the *parameter*, and `WHERE id = id` silently becomes always-true. That bug has deleted real data.

Parameters: IN, OUT, INOUT

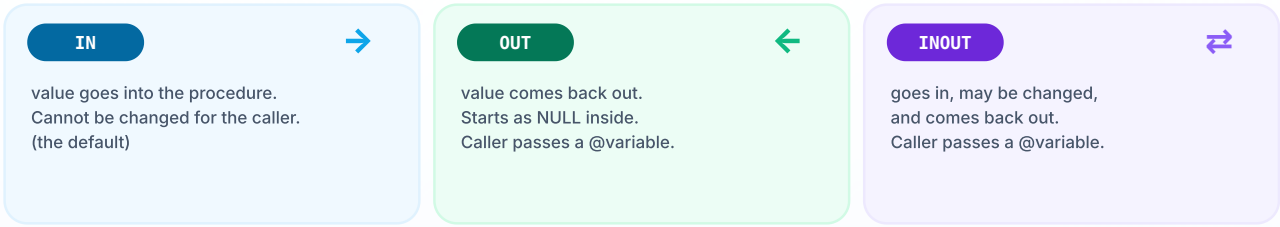


Figure 10. The three parameter modes.

SQL — `SELECT ... INTO` stores query results in variables

```sql
DELIMITER //
CREATE PROCEDURE category_stats(IN p_category VARCHAR(50), OUT p_count INT, OUT p_avg_price
DECIMAL(10,2))
BEGIN
    SELECT COUNT(*), ROUND(AVG(unit_price), 2)
    INTO   p_count, p_avg_price
    FROM   sales.dim_product
    WHERE  category = p_category;
END //

CREATE PROCEDURE bump(INOUT p_value INT)
BEGIN
    SET p_value = p_value + 10;
END //
DELIMITER ;
```

SQL — OUT values are read back through session variables (@name)

```sql
CALL category_stats('Toys', @n, @avg);
SELECT @n AS toy_products, @avg AS avg_price;
SET @v = 5;
CALL bump(@v);
SELECT @v AS after_bump;
```

OUT PARAMETERS OF CATEGORY_STATS — 1 ROW

toy_products	avg_price
78	519.18

INOUT PARAMETER AFTER BUMP — 1 ROW

after_bump
15

NOTE

`@name` variables are **user session variables**: they live as long as your connection and need no declaration. Variables declared with `DECLARE` inside a procedure are **local** and vanish when it ends.

Variables and control flow

Construct	Syntax	Use it for
Local variable	`DECLARE v INT DEFAULT 0;` then `SET v = v + 1;`	counters, intermediate values (declare first in the block)
Assign from a query	`SELECT COUNT(*) INTO v FROM t;`	capture a result
If / else	`IF c THEN … ELSEIF c2 THEN … ELSE … END IF;`	branching
Case	`CASE WHEN c THEN … ELSE … END CASE;`	many-way branching
While loop	`WHILE cond DO … END WHILE;`	loop while true
Repeat loop	`REPEAT … UNTIL cond END REPEAT;`	run at least once
Generic loop	`lbl: LOOP … LEAVE lbl; … END LOOP;`	custom exit with `LEAVE` / `ITERATE`

SQL — IF/ELSEIF branching and a WHILE loop

```sql
DELIMITER //
CREATE PROCEDURE classify_price(IN p_price DECIMAL(10,2), OUT p_label VARCHAR(10))
BEGIN
    IF p_price >= 800 THEN
        SET p_label = 'premium';
    ELSEIF p_price >= 300 THEN
        SET p_label = 'standard';
    ELSE
        SET p_label = 'budget';
    END IF;
END //

CREATE PROCEDURE fill_numbers(IN p_n INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    DROP TABLE IF EXISTS nums;
    CREATE TABLE nums (n INT, n_squared INT);
    WHILE i <= p_n DO
        INSERT INTO nums VALUES (i, i * i);
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;
```

```sql
CALL classify_price(897.35, @a);
CALL classify_price(150, @b);
SELECT @a AS price_897, @b AS price_150;
CALL fill_numbers(5);
SELECT * FROM nums;
```

IF / ELSEIF RESULT — 1 ROW

price_897	price_150
premium	budget

TABLE BUILT BY THE WHILE LOOP — 5 ROWS

n	n_squared
1	1
2	4
3	9
4	16
5	25

> ⚠ **WATCH OUT**
> Row-by-row loops are **slow** in SQL. If a plain `INSERT … SELECT` can do the job set-wise (which is almost always), use that. Loops belong to control logic (iterate over dates or tables), not to bulk data.

Error handling

A **handler** says: 'if this error happens, do that instead of crashing'. `SIGNAL` lets *you* raise an error, for example when a business rule is violated.

SQL — An EXIT HANDLER for a specific error number, and a custom error with SIGNAL

```sql
DELIMITER //
CREATE PROCEDURE add_customer_safe(IN p_id INT, IN p_name CHAR(100), IN p_email CHAR(100), OUT
p_status VARCHAR(40))
BEGIN
    DECLARE EXIT HANDLER FOR 1062            -- duplicate primary key
        SET p_status = 'duplicate id: skipped';
    INSERT INTO customers VALUES (p_id, p_name, p_email);
    SET p_status = 'inserted';
END //

CREATE PROCEDURE require_positive(IN p_amount DECIMAL(10,2))
BEGIN
    IF p_amount <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'amount must be positive';
    END IF;
END //
DELIMITER ;
```

```sql
CALL add_customer_safe(11, 'chen', 'chen@x.com', @s1);
CALL add_customer_safe(11, 'chen', 'chen@x.com', @s2);
SELECT @s1 AS first_call, @s2 AS second_call;
```

SECOND CALL HIT THE HANDLER INSTEAD OF FAILING — 1 ROW

first_call	second_call
inserted	duplicate id: skipped

SQL — SIGNAL raises a user-defined error

```sql
CALL require_positive(-5);
```

ERROR

```
ERROR 1644 (45000): amount must be positive
```

Transactions inside a procedure

Data jobs should be **all-or-nothing**. Start a transaction, do the changes, and let a handler roll everything back if any step fails (Phase 4 introduced transactions).

SQL — Atomic money transfer: both updates happen, or neither

```sql
DELIMITER //
CREATE PROCEDURE transfer(IN p_from INT, IN p_to INT, IN p_amount DECIMAL(10,2))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;               -- undo both updates
        RESIGNAL;               -- then report the original error
    END;

    START TRANSACTION;
    UPDATE accounts SET balance = balance - p_amount WHERE id = p_from;
    UPDATE accounts SET balance = balance + p_amount WHERE id = p_to;
    IF (SELECT balance FROM accounts WHERE id = p_from) < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'insufficient funds';
    END IF;
    COMMIT;
END //
DELIMITER ;
```

SQL — Successful transfer of 200

```sql
CALL transfer(1, 2, 200); SELECT * FROM accounts ORDER BY id;
```

RESULT — 2 ROWS

id	owner	balance
1	Asha	300.00
2	Bela	300.00

SQL — Not enough money: the procedure raises an error

```sql
CALL transfer(2, 1, 5000);
```

ERROR

```
ERROR 1644 (45000): insufficient funds
```

SQL — Balances unchanged after the failed call. The ROLLBACK undid the partial updates.

```sql
SELECT * FROM accounts ORDER BY id;
```

RESULT — 2 ROWS

id	owner	balance
1	Asha	300.00
2	Bela	300.00

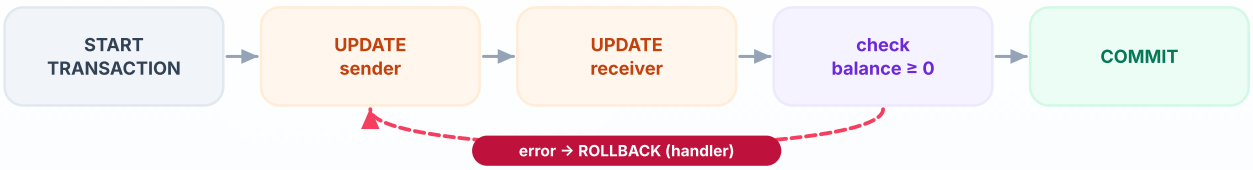


Figure 11. The happy path runs left to right. Any error jumps back and undoes the work.

An ETL-style procedure: idempotent daily refresh

Idempotent means 'safe to run twice': the second run produces the same final state as the first. The trick is **delete the target slice, then reload it**, inside one transaction.

SQL — Delete → reload → log, all in one transaction

```sql
DELIMITER //
CREATE PROCEDURE refresh_daily_sales(IN p_day DATE)
BEGIN
    DECLARE v_rows INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;
        DELETE FROM daily_summary WHERE day = p_day;            -- 1. clear the slice

        INSERT INTO daily_summary (day, orders, revenue, loaded_at)   -- 2. rebuild it
        SELECT p_day, t.orders, t.revenue, NOW()
        FROM  (SELECT COUNT(*) AS orders, COALESCE(SUM(f.total_amount), 0) AS revenue
               FROM   sales.fact_sales f
               JOIN   sales.dim_date d ON d.date_key = f.date_key
               WHERE  d.date = p_day) t;
        SET v_rows = ROW_COUNT();

        INSERT INTO etl_log (job, target_day, rows_loaded, finished_at)
        VALUES ('refresh_daily_sales', p_day, v_rows, NOW());   -- 3. audit trail
    COMMIT;
END //
DELIMITER ;
```

SQL — Two runs, still ONE summary row for the day: the load is idempotent

```sql
CALL refresh_daily_sales('2025-03-14');
CALL refresh_daily_sales('2025-03-14');          -- run it a second time
SELECT day, orders, revenue FROM daily_summary;
```

RESULT — 1 ROW

day	orders	revenue
2025-03-14	7	33080.93

SQL ...but the log shows both runs, which is what you want for auditing

```sql
SELECT run_id, job, target_day, rows_loaded FROM etl_log ORDER BY run_id;
```

RESULT 2 ROWS

run_id	job	target_day	rows_loaded
1	refresh_daily_sales	2025-03-14	1
2	refresh_daily_sales	2025-03-14	1

SQL A backfill wrapper: reload a whole date range, one idempotent day at a time

```sql
DELIMITER //
CREATE PROCEDURE backfill_daily_sales(IN p_from DATE, IN p_to DATE)
BEGIN
    DECLARE d DATE DEFAULT p_from;
    WHILE d <= p_to DO
        CALL refresh_daily_sales(d);          -- procedures can call procedures
        SET d = d + INTERVAL 1 DAY;
    END WHILE;
END //
DELIMITER ;
```

SQL Five days loaded (the 14th was safely reloaded)

```sql
CALL backfill_daily_sales('2025-03-10', '2025-03-14'); SELECT day, orders, revenue FROM
daily_summary ORDER BY day;
```

RESULT 5 ROWS

day	orders	revenue
2025-03-10	7	15923.52
2025-03-11	10	18076.70
2025-03-12	3	6727.91
2025-03-13	10	24830.44
2025-03-14	7	33080.93

DATA-ENGINEERING LENS

Idempotency + backfill = reliable pipelines. Schedulers retry failed jobs and engineers re-run history after bug fixes. If every run replaces its own partition (a day, an hour), re-running never double-counts. Airflow, dbt incremental models and Spark `overwrite` partitions all use this idea.

Managing procedures

Task	Statement
Run	`CALL name(args);`
Inspect	`SHOW CREATE PROCEDURE name;` · `SHOW PROCEDURE STATUS WHERE Db = 'lab';`
Change	drop and re-create: `DROP PROCEDURE IF EXISTS name;`

Strengths	Trade-offs
Fewer network round trips; logic runs next to the data	Business logic hidden in the database is harder to version-control, test and review
One definition shared by every client	Debugging tools are weaker than in Python or Java
Permissions: grant `EXECUTE`, not table access	Tied to one database vendor's dialect (not portable)

REMEMBER

Most teams now keep transformation logic as **SQL files in git** run by an orchestrator. You still need to read and write procedures: they are everywhere in existing systems and in interviews.

7.4 PART 7.4 · YOUR OWN SCALAR FUNCTIONS

User-Defined Functions

A **user-defined function** (UDF) is a stored routine that takes values and **returns exactly one value**, so you can use it inside any expression: `SELECT square_it(price) …`. It is the SQL equivalent of writing your own `ROUND()` or `UPPER()`.

What you will learn

- Create a function with `RETURNS` and `RETURN`
- Choose between `DETERMINISTIC`, `READS SQL DATA` and friends
- Spot the integer-parameter rounding trap
- Decide between a function and a procedure
- Know the performance cost of calling a function per row

Why it matters for data engineering. Reusable cleaning rules (normalize a phone number, bucket a price, build a surrogate key) are natural functions. They make queries read like the business ('`fn_price_band(price)`') and keep each rule defined in one place.

Procedure or function?

	Stored procedure	User-defined function
Called with	`CALL p(…)` as its own statement	inside any expression: `SELECT f(x)`, `WHERE f(x) > 1`
Returns	nothing, or OUT params, or result sets	**exactly one value** (`RETURNS type`)
Parameters	`IN`, `OUT`, `INOUT`	`IN` only
Can change data (INSERT/UPDATE)	yes	technically possible, but discouraged
Can use transactions (`COMMIT`)	yes	no
Typical job	run a multi-step job	compute a value for each row

The course's function

SQL — From the course (`ch-14`)

```sql
DELIMITER //
CREATE FUNCTION square_it(x INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN x * x;
END //
DELIMITER ;
```

SQL — It works... but look closely at the numbers

```sql
SELECT unit_price, square_it(unit_price) AS squared FROM sales.dim_product LIMIT 3;
```

RESULT — 3 ROWS

unit_price	squared
897.35	804609
920.58	848241
553.56	306916



The INT parameter silently rounded the price before squaring: the answer is off by hundreds.

Figure 12. A parameter's declared type is applied to the argument, before your code runs.

SQL — Fix: declare types that fit the data

```sql
DELIMITER //
CREATE FUNCTION square_dec(x DECIMAL(12,2))
RETURNS DECIMAL(24,4)
DETERMINISTIC
BEGIN
    RETURN x * x;
END //
DELIMITER ;
```

SQL — Side by side: the DECIMAL version is exact

```sql
SELECT unit_price, square_it(unit_price) AS int_version, square_dec(unit_price) AS decimal_version
FROM sales.dim_product LIMIT 3;
```

RESULT — 3 ROWS

unit_price	int_version	decimal_version
897.35	804609	805237.0225
920.58	848241	847467.5364
553.56	306916	306428.6736

WATCH OUT

Match parameter and return types to your data. `INT` rounds decimals, `VARCHAR(10)` truncates longer text (or errors in strict mode). Mismatched types are the most common UDF bug.

Function characteristics

Keyword	Promise you make to MySQL
`DETERMINISTIC`	same inputs → always the same output (no `NOW()`, `RAND()`)
`NOT DETERMINISTIC`	the output may differ for the same inputs (default if you say nothing)
`NO SQL`	contains no SQL statements, just expressions
`READS SQL DATA`	runs `SELECT`s but changes nothing
`MODIFIES SQL DATA`	may change data

NOTE

When **binary logging** is on (typical for replicated servers), MySQL refuses to create a function that declares none of `DETERMINISTIC`, `NO SQL` or `READS SQL DATA` (error 1418). Declaring the truth is therefore both a correctness and a deployment matter.

Useful everyday functions

SQL — A business rule and a cleaning rule, each defined once

```sql
DELIMITER //
CREATE FUNCTION fn_price_band(p DECIMAL(10,2))
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    RETURN CASE WHEN p >= 800 THEN 'premium'
                WHEN p >= 300 THEN 'standard'
                ELSE 'budget' END;
END //

CREATE FUNCTION fn_clean_phone(p VARCHAR(30))
RETURNS CHAR(10)
DETERMINISTIC
BEGIN
    RETURN RIGHT(REGEXP_REPLACE(p, '[^0-9]', ''), 10);   -- keep digits, last 10
END //
DELIMITER ;
```

SQL — A function inside GROUP BY: the query reads like the business question

```sql
SELECT fn_price_band(unit_price) AS band, COUNT(*) AS products, ROUND(AVG(unit_price)) AS avg_price
FROM   sales.dim_product
GROUP  BY fn_price_band(unit_price)
ORDER  BY avg_price DESC;
```

RESULT — 3 ROWS

band	products	avg_price
premium	93	895
standard	250	560
budget	157	157

SQL — Normalizing messy phone numbers (the data-quality issue from Phase 3, solved with one function)

```sql
SELECT phone AS raw_phone, fn_clean_phone(phone) AS phone_10
FROM   sales.dim_customer
ORDER  BY LENGTH(phone) DESC, customer_key
LIMIT  4;
```

RESULT — 4 ROWS

raw_phone	phone_10
001596505299143	6505299143
182656936555456	6936555456
001691509683873	1509683873
317474030162051	4030162051

Performance: functions run once per row

✓ Function on the output (SELECT)

```
SELECT fn_clean_phone(phone) FROM t
```

Runs after filtering, once per returned row.
Indexes still do their job.

× Function on the column (WHERE)

```
WHERE fn_clean_phone(phone) = '5551234567'
```

Computed for EVERY row, and the index on
`phone` cannot be used (not SARGable, Phase 5).

Figure 13. Where you call a function matters as much as what it does.

INTERVIEW ANGLE

"Procedure vs function?" A function returns one value and is used inside queries; a procedure is called on its own, can have OUT parameters, run multi-statement logic and control transactions. "Why can a function slow a query?" It is evaluated for every row, and wrapping a column in it blocks index use.

Managing functions

Task	Statement
List	`SHOW FUNCTION STATUS WHERE Db = 'lab';`
See the code	`SHOW CREATE FUNCTION name;`
Remove	`DROP FUNCTION IF EXISTS name;`

7.5 PART 7.5 · FROM SQL TO DATA ENGINEERING

The Next Steps

You now speak SQL fluently: you can design tables, load and clean data, answer analytical questions and package logic for reuse. This final part shows **where SQL sits in a data platform**, gives you a taste of the next big SQL topic (**indexes**), builds a small **end-to-end pipeline** from everything you learned, and lays out a roadmap.

What you will learn

- Place SQL inside a modern data platform
- Read a basic `EXPLAIN` plan and see the effect of an index
- Assemble raw → staging → mart using views, CTEs, windows and a procedure
- Follow a staged learning path toward a data-engineering role

Why it matters for data engineering. Knowing the whole picture is what turns a SQL user into a data engineer. Every tool in the roadmap (Python, Airflow, dbt, Spark, warehouses) either *runs* SQL, *schedules* SQL, or *moves data to and from* a SQL system.

Where SQL sits in a data platform

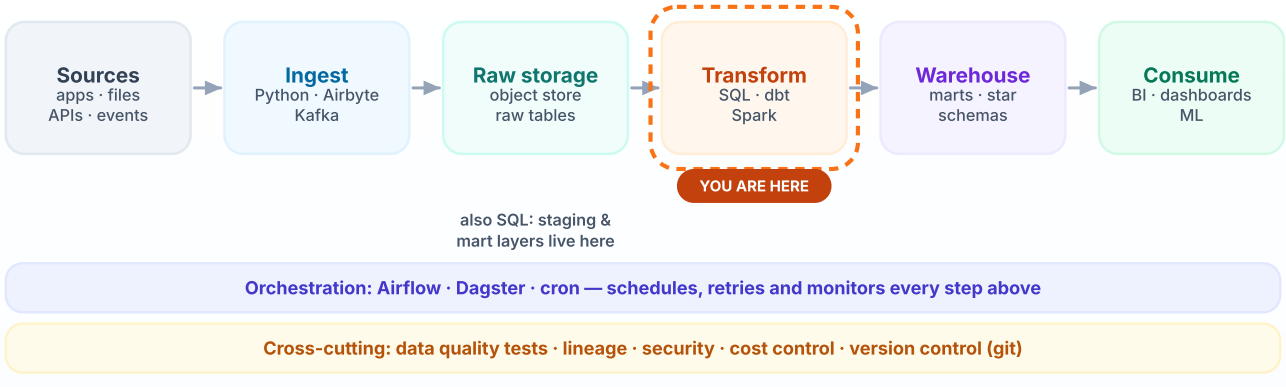


Figure 14. A simplified data platform. The orange area is where the skills from these seven phases are used every day.

A taste of indexes and EXPLAIN

An **index** is a sorted lookup structure (a B-tree) that lets MySQL jump to matching rows instead of reading the whole table, like a book's index versus reading every page. `EXPLAIN` shows the plan MySQL intends to use.

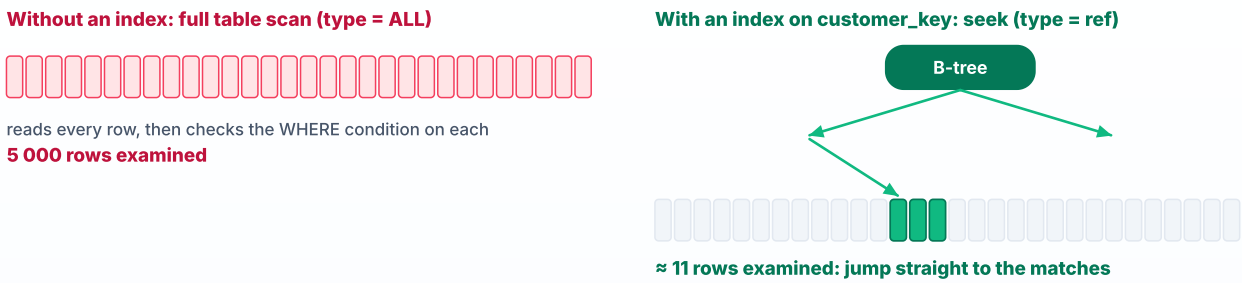


Figure 15. Same question, very different work. The plan numbers below are from a real run.

SQL — No index yet: `type = ALL`, 5000 rows examined

```sql
EXPLAIN SELECT * FROM sales_copy WHERE customer_key = 42;
```

RESULT — 1 ROW

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	sales_copy	ALL	*NULL*	*NULL*	*NULL*	*NULL*	5000	Using where

SQL — After adding the index: `type = ref`, using `idx_customer`, about 11 rows examined

```sql
ALTER TABLE sales_copy ADD INDEX idx_customer (customer_key); EXPLAIN SELECT * FROM sales_copy WHERE customer_key = 42;
```

RESULT — 1 ROW

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	sales_copy	ref	idx_customer	idx_customer	5	const	11	

Index rule of thumb	Why
Index columns used in `WHERE`, `JOIN … ON` and `ORDER BY`	these are the operations an index can speed up
For a multi-column index, put the equality column first (**left-most prefix** rule)	`INDEX(a, b)` helps `WHERE a = …` and `a = … AND b = …`, but not `WHERE b = …` alone
Keep indexed columns bare in conditions	`WHERE YEAR(d) = 2024` cannot use an index on `d` (Phase 5)
Do not index everything	each index costs disk and slows `INSERT` / `UPDATE` / `DELETE`
Primary and unique keys are indexes already	Phase 2: the engine builds them to enforce the rule

> **NOTE**
> Analytical warehouses (Snowflake, BigQuery, Redshift) mostly **do not use B-tree indexes**. They rely on columnar storage, partitioning and clustering instead. The *idea* (organize data so queries read less of it) is the same, which is why learning `EXPLAIN` and 'reading less data' transfers.

Capstone: a mini ELT pipeline

Let's combine the whole course into one small, realistic pipeline over the sales warehouse: **clean** the raw facts in a staging view, **summarize** them into a mart table with a CTE and window functions, and **refresh** the mart with an idempotent stored procedure.

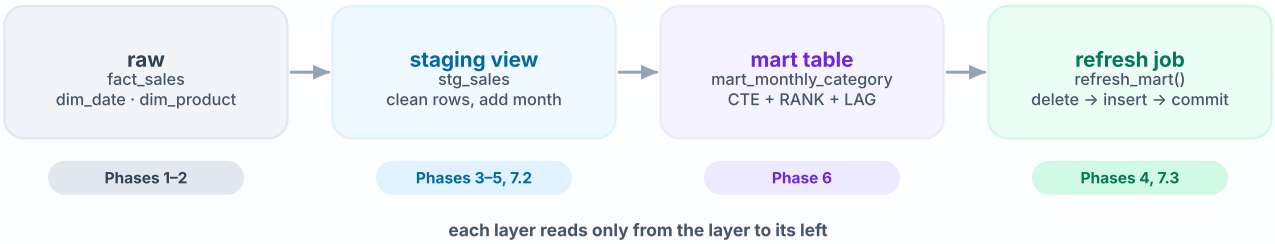


Figure 16. The pipeline we are about to build, and which phases each layer draws on.

Step 1 · Staging view

SQL — Clean, renamed, filtered: this is the only object analysts should query

```sql
CREATE VIEW stg_sales AS
SELECT f.sales_id,
       d.date                          AS sale_date,
       DATE_FORMAT(d.date, '%Y-%m')    AS ym,
       p.category,
       f.quantity_sold,
       f.total_amount
FROM   sales.fact_sales  f
JOIN   sales.dim_date    d ON d.date_key    = f.date_key
JOIN   sales.dim_product p ON p.product_key = f.product_key
WHERE  f.total_amount >= 0;                -- drop the 6 rows with negative totals (data-quality rule)
```

SQL — Six bad rows were excluded (Phase 3's profiling discovered them)

```sql
SELECT (SELECT COUNT(*) FROM sales.fact_sales) AS raw_rows, (SELECT COUNT(*) FROM stg_sales) AS staged_rows;
```

RESULT — 1 ROW

raw_rows	staged_rows
5000	4994

Step 2 · Mart table and refresh procedure

SQL — DDL (Phase 2) + procedure (7.3) + CTE and window functions (Phase 6)

```sql
CREATE TABLE mart_monthly_category (
    ym            CHAR(7),
    category      VARCHAR(50),
    orders        INT,
    revenue       DECIMAL(14,0),
    rank_in_month INT,
    mom_pct       DECIMAL(6,1),
    PRIMARY KEY (ym, category)
);

DELIMITER //
CREATE PROCEDURE refresh_mart()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;
    START TRANSACTION;
        DELETE FROM mart_monthly_category;
        INSERT INTO mart_monthly_category
        WITH m AS (
            SELECT ym, category, COUNT(*) AS orders, ROUND(SUM(total_amount)) AS revenue
            FROM   stg_sales
            GROUP  BY ym, category
        )
        SELECT ym, category, orders, revenue,
               RANK() OVER (PARTITION BY ym ORDER BY revenue DESC),
               ROUND(100 * (revenue - LAG(revenue) OVER (PARTITION BY category ORDER BY ym))
                         / LAG(revenue) OVER (PARTITION BY category ORDER BY ym), 1)
        FROM   m;
    COMMIT;
END //
DELIMITER ;
```

SQL — Run it twice: still 150 rows (6 categories × 25 months). Idempotent.

```sql
CALL refresh_mart(); CALL refresh_mart(); SELECT COUNT(*) AS mart_rows FROM mart_monthly_category;
```

RESULT 1 ROW

mart_rows
150

Step 3 · Ask the business question

SQL — "Which category sold the most each month, and how did it change?" is now a trivial query

```sql
SELECT ym, category AS top_category, revenue, mom_pct
FROM   mart_monthly_category
WHERE  rank_in_month = 1
ORDER  BY ym
LIMIT  8;
```

RESULT — 8 ROWS

ym	top_category	revenue	mom_pct
2023-10	Clothing	86335	*NULL*
2023-11	Books	113897	105.2
2023-12	Clothing	143112	47.1
2024-01	Toys	143542	102.6
2024-02	Clothing	123421	14.2
2024-03	Home & Kitchen	135743	34.4
2024-04	Books	127473	54.9
2024-05	Sports	116702	53.3

DATA-ENGINEERING LENS

What you just built is dbt in miniature: a staging model (a view), a mart model (a table built from a SELECT with a CTE) and a rebuild job. Add a scheduler to run `refresh_mart()` nightly, and tests that assert `rank_in_month` is never NULL, and you have a real pipeline. Try swapping in your own dataset!

Your roadmap

▲ skills grow step by step; each stage builds on the previous one

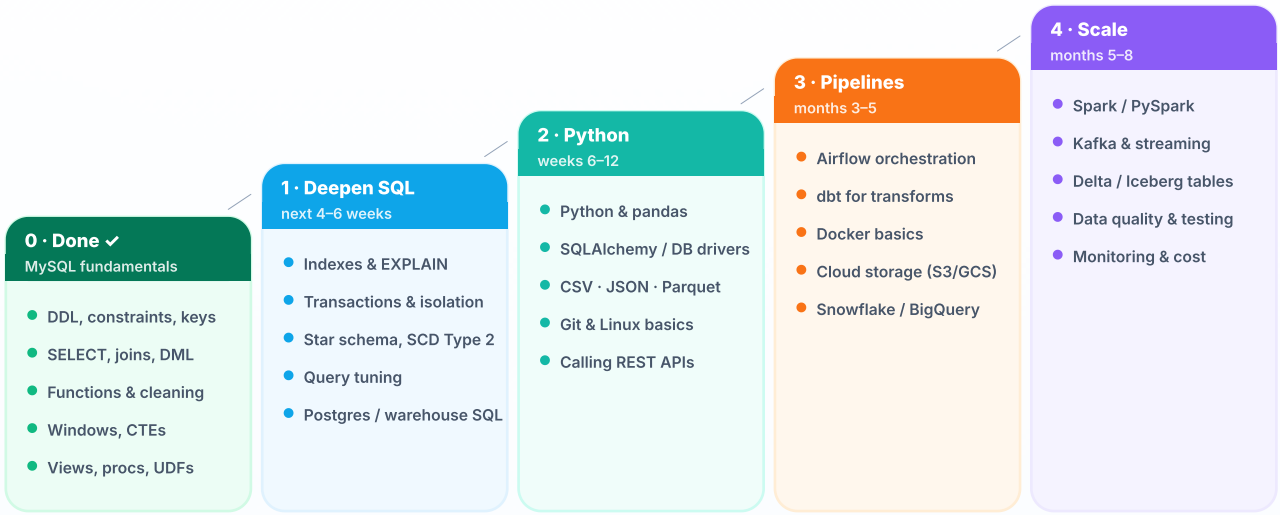


Figure 17. A staircase roadmap. Stage 1 is the natural continuation of this course, and stages 2 and 3 are where most junior data-engineering roles start.

When	Goal	Concrete deliverable
Weeks 1–2	Repeat Phases 3–6 on a **new** dataset	20 interview-style queries solved without notes
Weeks 3–4	Indexes, `EXPLAIN`, transactions	a before/after tuning write-up on a 1M-row table
Weeks 5–6	Dimensional modeling	a star schema for a domain you like, loaded by SQL
Weeks 7–9	Python + SQL	a script that loads a CSV into MySQL and runs your mart refresh
Weeks 10–12	Orchestration + dbt	an Airflow DAG (or cron) that runs it daily, with tests

Interview question bank

Fundamentals and design

Q1 What is the difference between `DELETE`, `TRUNCATE` and `DROP`?

`DELETE` removes chosen rows (logged, can be rolled back, keeps the table). `TRUNCATE` empties the whole table quickly and resets AUTO_INCREMENT (DDL, cannot be rolled back). `DROP` removes the table itself.

Q2 Primary key vs unique key?

Both enforce uniqueness. A table has one primary key (never NULL); it can have many unique keys (which may allow NULLs).

Q3 What is a foreign key and why use one?

A column that references another table's key. It enforces referential integrity, so you cannot create orphan rows.

Q4 What is normalization?

Organizing tables to remove redundancy (1NF → 3NF) so each fact is stored once. Warehouses often *denormalize* deliberately for read speed.

Q5 Explain the logical order of a SELECT.

FROM/JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT → DISTINCT → ORDER BY → LIMIT.

Querying

Q1 `WHERE` vs `HAVING`?

`WHERE` filters rows before grouping; `HAVING` filters groups after aggregation.

Q2 `INNER JOIN` vs `LEFT JOIN`?

INNER keeps only matches. LEFT keeps every left row, with NULLs where the right side has no match.

Q3 How do you find rows in A that have no match in B?

`LEFT JOIN … WHERE b.key IS NULL`, or `NOT EXISTS`. Avoid `NOT IN` if the column can contain NULL.

Q4 `UNION` vs `UNION ALL`?

UNION removes duplicates (extra sort/hash); UNION ALL keeps everything and is faster.

Q5 How does `COUNT(*)` differ from `COUNT(col)`?

`COUNT(*)` counts rows; `COUNT(col)` skips NULLs; `COUNT(DISTINCT col)` counts unique non-NULL values.

Q6 Why is `WHERE YEAR(date) = 2024` slow?

The function on the column prevents index use. Use a range: `date >= '2024-01-01' AND date < '2025-01-01'`.

Advanced

Q1 `ROW_NUMBER` vs `RANK` vs `DENSE_RANK`?

ROW_NUMBER is unique; RANK gives ties the same number then skips; DENSE_RANK gives ties the same number without gaps.

Q2 Top-N per group?

Rank with a window function in a subquery/CTE, then filter `rn <= N` outside.

Q3 How do you remove duplicates?

`ROW_NUMBER() OVER (PARTITION BY key ORDER BY …)` and keep 1, into a new table or view.

Q4 CTE vs subquery vs view vs temp table?

CTE: named step in one statement. Subquery: inline. View: stored query. Temp table: stored data for the session.

Q5 What is a correlated subquery?

An inner query that references the outer row, evaluated per outer row. Often rewritten with a join or a window function.

Q6 What does idempotent mean for a pipeline?

Running it twice gives the same final state as running it once (e.g., delete the partition, then reload it in a transaction).

Q7 Procedure vs function?

Function returns one value and is used in expressions. Procedure is `CALL`ed, can have OUT params, multi-statement logic and transactions.

Q8 What is a view and is it faster than the query?

A stored SELECT with no data of its own. Not faster by itself; it simplifies and standardizes. A summary table is the 'materialized' alternative.

Master cheat sheet

Phase	Remember
1 · Foundations	SQL is declarative · RDBMS = tables, rows, columns, keys · 5 sublanguages: DDL, DML, DQL, DCL, TCL
2 · Design	`CREATE / ALTER / DROP / TRUNCATE` · `NOT NULL UNIQUE CHECK DEFAULT` · PK + FK · surrogate vs natural keys · normalization
3 · Querying	`SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT` · logical order ≠ written order · NULL needs `IS NULL`
4 · Joins & DML	INNER · LEFT · RIGHT · CROSS · self · anti-join · `INSERT / UPDATE / DELETE` · upsert · transactions `START … COMMIT / ROLLBACK`
5 · Functions	numeric · string · date · `CAST` · `CASE` · `COALESCE / NULLIF` · conditional aggregation · clean layer

Phase	Remember
6 · Analytics	`OVER(PARTITION BY … ORDER BY … frame)` · ranking · `LAG/LEAD` · subqueries · `EXISTS` · CTE · recursive CTE
7 · Production	dedup · n-th value · gaps & islands · views · `CALL` procedures · handlers · UDFs · idempotent loads · indexes & `EXPLAIN`

Course topic coverage map

Course topic	Where in these notes
Introduction · What is SQL and MySQL? · Environment setup	Phase 1 (Parts 1.1–1.3)
DDL commands · Constraints · Keys	Phase 2 (Parts 2.1–2.3)
SQL basic commands · Execution flow	Phase 3 (Parts 3.1–3.2)
SQL joins · DML commands	Phase 4 (Parts 4.1–4.2)
Numeric, string and date functions	Phase 5 (Parts 5.1–5.6)
Window functions · Ranking functions · Subqueries · CTEs	Phase 6 (Parts 6.1–6.5)
SQL real-time scenarios	Phase 7 (Part 7.1)
Views · Stored procedures · User-defined functions	Phase 7 (Parts 7.2–7.4)
The next steps	Phase 7 (Part 7.5)

Keep practicing

- The **MySQL 8.0 Reference Manual** (the official docs): bookmark the pages for window functions, CTEs and `EXPLAIN`.
- **Interview-style practice:** LeetCode's SQL set, StrataScratch and DataLemur offer graded problems on exactly the patterns of Phase 6 and 7.1.
- **Books and guides:** *Use The Index, Luke* (indexing), *The Data Warehouse Toolkit* (dimensional modeling), the dbt documentation (analytics engineering).
- **Build, don't just read:** pick a public dataset, create raw → staging → mart as in the capstone, and put the SQL on GitHub. A small portfolio project is worth more than another course.

7.6 WRAP-UP

Phase 7 Recap & Practice

Check the key ideas of production SQL, then practice with five tasks that build a view, a function and a procedure, and apply two scenario patterns.

Cheat sheet

Topic	Pattern
n-th per group	`DENSE_RANK() OVER (PARTITION BY g ORDER BY x DESC)` in a subquery, `WHERE rank = n`
Deduplicate	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY ts DESC)` then `WHERE rn = 1`
Previous / next row	`LAG(x, 1, default) OVER (ORDER BY d)` · `LEAD(...)`
Streaks	`date − ROW_NUMBER()` constant inside a streak → `GROUP BY` that
View	`CREATE [OR REPLACE] VIEW v AS SELECT …` · live, no data · `WITH CHECK OPTION`
Procedure	`DELIMITER //` · `CREATE PROCEDURE p(IN … OUT …) BEGIN … END //` · `CALL p(…)`
Robust procedure	`START TRANSACTION` + `EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END` + `SIGNAL`
Function	`CREATE FUNCTION f(x TYPE) RETURNS TYPE DETERMINISTIC BEGIN RETURN …; END`
Idempotent load	delete the slice → reload it → commit, in one transaction
Index check	`EXPLAIN SELECT …`: want `type = ref/range`, not `ALL`

Check yourself

Q1 Why does `WHERE rn = 1` need a subquery or CTE?

Window functions cannot appear in `WHERE`; compute the rank in an inner query, filter in the outer one.

Q2 Does a view store data?

No, only the SELECT text. Each query on the view re-runs it against the base tables.

Q3 What does `WITH CHECK OPTION` prevent?

Changes made through the view that would make the row no longer satisfy the view's own `WHERE`.

Q4 Why change the delimiter when creating a procedure?

So the client does not end the statement at the first `;` inside the body.

Q5 What does `INOUT` mean?

The caller passes a value in, the procedure may modify it, and the new value is visible to the caller afterwards.

Q6 Why can a UDF with an INT parameter give wrong results on prices?

The decimal argument is rounded to INT before your code runs. Declare DECIMAL parameters.

Q7 What makes a load idempotent?

Re-running replaces its own slice (delete + insert in a transaction), so the result does not depend on how many times it ran.

Practice lab

1. Create a view `v_big_sales` of `sales.fact_sales` rows with `total_amount >= 9000`, showing `sales_id`, `date_key`, `total_amount`. How many rows does it return?
2. Write a function `fn_gst(amount DECIMAL(12,2))` that returns 18 % of the amount rounded to 2 decimals. Show it for the 3 largest sales.
3. Write a procedure `sales_in_range(IN p_from INT, IN p_to INT)` (using `date_key`) that returns the number of sales and the total revenue in that range. Call it.
4. Using `weather`, list the days that were **warmer than the previous day**.
5. For each customer, find the **most recent purchase** (`customer_key`, `date_key`, `total_amount`). Show 3 rows.

Answer key

SQL 1

```sql
CREATE VIEW v_big_sales AS
SELECT sales_id, date_key, total_amount FROM sales.fact_sales WHERE total_amount >= 9000;
```

SQL

```sql
SELECT COUNT(*) AS big_sales FROM v_big_sales;
```

RESULT 1 ROW

big_sales
46

SQL 2

```sql
DELIMITER //
CREATE FUNCTION fn_gst(amount DECIMAL(12,2)) RETURNS DECIMAL(12,2) DETERMINISTIC
BEGIN
    RETURN ROUND(amount * 0.18, 2);
END //
DELIMITER ;
```

SQL

```sql
SELECT sales_id, total_amount, fn_gst(total_amount) AS gst FROM sales.fact_sales ORDER BY
total_amount DESC LIMIT 3;
```

RESULT 3 ROWS

sales_id	total_amount	gst
2204	9949.60	1790.93
3251	9937.39	1788.73
3745	9933.24	1787.98

SQL 3

```sql
DELIMITER //
CREATE PROCEDURE sales_in_range(IN p_from INT, IN p_to INT)
BEGIN
    SELECT COUNT(*) AS sales, ROUND(SUM(total_amount)) AS revenue
    FROM   sales.fact_sales
    WHERE  date_key BETWEEN p_from AND p_to;
END //
DELIMITER ;
```

SQL

```sql
CALL sales_in_range(20240101, 20240131);
```

RESULT 1 ROW

sales	revenue
236	690291

SQL 4

```sql
SELECT record_date, temp, prev_temp
FROM (SELECT *, LAG(temp) OVER (ORDER BY record_date) AS prev_temp FROM weather) t
WHERE temp > prev_temp;
```

RESULT 2 ROWS

record_date	temp	prev_temp
2025-01-02	25	10
2025-01-04	30	20

SQL 5

```sql
SELECT customer_key, date_key, total_amount
FROM (
    SELECT customer_key, date_key, total_amount,
           ROW_NUMBER() OVER (PARTITION BY customer_key ORDER BY date_key DESC, sales_id DESC) AS rn
    FROM   sales.fact_sales
) t
WHERE rn = 1
ORDER BY customer_key
LIMIT 3;
```

RESULT 3 ROWS

customer_key	date_key	total_amount
1	20250902	8851.41
2	20250414	2315.66
3	20250708	1020.88

FINISH LINE

You have completed the course material. The path from here is practice on new data, then Python and an orchestrator. Pick one small project this week and build it end to end. Good luck on the data-engineering journey!